

LAPORAN PRAKTIKUM STRUKTUR DATA

Double LinkedList



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 12 Mei 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Double LinkedList”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Double LinkedList. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah struktur data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 12 Mei 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah Pengerjaan	3
BAB III KESIMPULAN.....	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data yang efisien dan terstruktur merupakan fondasi utama dalam pengembangan perangkat lunak modern. Seiring meningkatnya kompleksitas sistem, pengembang tidak hanya dituntut memahami cara menyimpan data, tetapi juga mampu memisahkan logika penggunaan dari detail implementasi. Konsep *Abstract Data Type* (ADT) hadir sebagai solusi melalui prinsip abstraksi yang mendefinisikan tipe data berdasarkan perilaku dan operasi yang diizinkan, bukan representasi memorinya. Dengan memisahkan antarmuka (*interface*) dari implementasi, ADT mendukung *information hiding*, enkapsulasi, serta membentuk sistem yang modular, *reusable*, dan mudah diuji.

Meskipun bersifat teoretis, pemahaman ADT memerlukan implementasi langsung untuk menguji kebenaran operasi, menangani kondisi batas (*boundary conditions*), dan memvalidasi kontrak spesifikasi yang ditetapkan. Penguasaan konsep ini juga menjadi prasyarat penting sebelum mempelajari algoritma lanjutan maupun arsitektur perangkat lunak berbasis komponen. Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pengalaman langsung dalam merancang, mengimplementasikan, dan menguji ADT sesuai spesifikasi teknis.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur ADT Double LinkedList
2. Memahami langkah-langkah menulis ADT Double LinkedList

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang ADT Double LinkedList
2. Meningkatkan pemahaman bagaimana cara menulis ADT Double LinkedList

BAB II

PEMBAHASAN

2.1 Landasan teori

Struktur data adalah cara penyimpanan, pengorganisasian, dan pengaturan data di dalam media penyimpanan komputer sehingga data tersebut dapat digunakan secara efisien. Sedangkan data adalah representasi dari fakta dunia nyata. Fakta atau keterangan tentang kenyataan yang disimpan, direkam atau direpresentasikan dalam bentuk tulisan, suara, gambar, sinyal atau simbol.

Definisi ADT (Abstract Data Type) sendiri dalam struktur data merupakan sebuah spesifikasi atau kontrak yang mendefinisikan antarmuka suatu tipe data. ADT tidak menentukan bagaimana tipe data diimplementasikan, melainkan berfokus pada operasinya dan perilaku yang diharapkan dari tipe data tersebut.

Double Linked List (sering disebut Doubly Linked List) adalah struktur data linear yang terdiri dari rangkaian node (simpul) yang saling terhubung. Berbeda dengan Singly Linked List yang hanya memiliki satu penunjuk ke node berikutnya, setiap node pada Double Linked List menyimpan dua penunjuk (pointer):

- next : mengarah ke node setelahnya
- prev : mengarah ke node sebelumnya

2.2 Langkah Pengerjaan

- NodeDLL_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan6_2511532017 dan class NodeDLL_2511532017.
 2. Setelah dibuat, kita akan membuat implementasi dasar Node

```
package pekan6_2511532017;

public class NodeDLL_2511532017 {
    //mendefinisikan kelas node
    int data_2017; //data
    NodeDLL_2511532017 next_2017; //pointer ke next node
    NodeDLL_2511532017 prev_2017; //pointer ke prev node
}
```

3. Disini data_2017 mendefinisikan sebuah kelas dengan tipe int, ini berfungsi untuk menyimpan variabel yang dibawa oleh node
4. NodeSLL_2511532017 next_2017 digunakan mendeklarasikan variabel referensi yang bertipe sama dengan kelasnya sendiri, ini merupakan pointer atau penghubung yang menunjukkan ke node berikutnya, lalu NodeDLL_2511532017 prev_2017 untuk kembali ke node sebelumnya

```
//konstruktor
public NodeDLL_2511532017 (int data_2017) {
    this.data_2017 = data_2017;
    this.next_2017 = null;
    this.prev_2017 = null;
}
}
```

5. Ini merupakan sebuah constructor, public NodeDLL_2511532017(int data_2017) akan menerima satu parameter integer yang akan menjadi isi data node, lalu this.data_2017 = data_2017; this disini merujuk pada variabel anggota kelas. this.next_2017 = null; berguna mengatur penghubung ke node berikutnya dan this.prev_2017 = null; untuk meangatur penghubung ke node sebelumnya

- InsertDLL_2511532017

1. Kita akan membuat class baru untuk InsertDLL_2511532017

```
package pekan6_2511532017;

public class InsertDLL_2511532017 {
    // menambahkan node di awal DLL
    static NodeDLL_2511532017 insertBegin_2017 (NodeDLL_2511532017 head_2017, int data_2017) {
        //buat node baru
        NodeDLL_2511532017 new_node = new NodeDLL_2511532017 (data_2017);
        //jadikan next pointer head
        new_node.next_2017 = head_2017;
        //jadikan pointer prev head ke new_node
        if (head_2017 != null) {
            head_2017.prev_2017 = new_node;
        }
        return new_node;
    }
}
```

2. insertBegin_2017 digunakan untuk buat node baru, new_node.next_2017 = head_2017 digunakan sebagai pointer ke node baru, jika list tidak kosong pointer prev dari head lama kini menagarah ke node baru, lalu return new_node

```

//fungsi untuk menampilkan node di akhir
public static NodeDLL_2511532017 InsertEnd_2017 (NodeDLL_2511532017 head_2017, int newData_2017) {
    //buat node baru
    NodeDLL_2511532017 newNode_2017 = new NodeDLL_2511532017 (newData_2017);
    //jika dll null jadikan head
    if (head_2017 == null) {
        head_2017 = newNode_2017;
    }
    else {
        NodeDLL_2511532017 curr_2017 = head_2017;
        while (curr_2017.next_2017 != null ) {
            curr_2017 = curr_2017.next_2017;
        }
        curr_2017.next_2017 = newNode_2017;
        newNode_2017.prev_2017 = curr_2017;
    }
    return head_2017;
}

```

3. InsertEnd_2017, jika list kosong, node baru langsung menjadi head, curr_2017 = head_2017 merupakan inisialisasi pointer untuk traversing, loop hingga menemukan node terakhir, lalu hubungkan node terakhir ke node baru dan set pointer prev node baru ke node sebelumnya

```

public static NodeDLL_2511532017 InsertAtPosition_2017 (NodeDLL_2511532017 head_2017, int pos_2017, int new_data_2017) {
    //buat node baru
    NodeDLL_2511532017 new_node_2017 = new NodeDLL_2511532017 (new_data_2017);
    if(pos_2017 == 1) {
        new_node_2017.next_2017 = head_2017;
        if (head_2017 == null) {
            head_2017.prev_2017 = new_node_2017;
        }
        head_2017 = new_node_2017;
        return head_2017;
    }
    NodeDLL_2511532017 curr_2017 = head_2017;
    for(int i = 1; i < pos_2017; i && curr_2017 != null; ++i) {
        curr_2017 = curr_2017.next_2017;
    }
    if (curr_2017 == null) {
        System.out.println("Posisi tidak ada");
        return head_2017;
    }
    new_node_2017.prev_2017 = curr_2017;
    new_node_2017.next_2017 = curr_2017.next_2017;
    curr_2017.next_2017 = new_node_2017;
    if (new_node_2017.next_2017 != null) {
        new_node_2017.next_2017.prev_2017 = new_node_2017;
    }
    return head_2017;
}

```

4. insertAtPosition_2017 , if (pos_2017 == 1) untuk ngehandle jika insert di awal list, loop geser pointer curr_2017 ke node sebelum posisi target, jika posisi melebihi list batalkan operasi, new_node.prev = curr untuk set pointer prev node baru ke node saat ini, new_node.next = curr.next set pointer next node baru ke node setelah cur, curr.next = new_node hubungkan curr ke node baru

```

public static void printList_2017 (NodeDLL_2511532017 head_2017) {
    NodeDLL_2511532017 curr_2017 = head_2017;
    while (curr_2017 != null) {
        System.out.print(curr_2017.data_2017 + " <--> ");
        curr_2017 = curr_2017.next_2017;
    }
    System.out.println();
}

```

5. printList_2017, curr_2017 = head_2017 mulai traversing dari node pertama, loop selama masih ada node yang valid, dan cetak dengan tanda <-->, curr_2017 = curr_2017.next_2017 untuk pindah ke node berikutnya

```

public static void main (String [] args) {
    //membuat dll
    NodeDLL_2511532017 head_2017 = new NodeDLL_2511532017 (2);
    head_2017.next_2017 = new NodeDLL_2511532017 (3);
    head_2017.next_2017.prev_2017 = head_2017;
    head_2017.next_2017.next_2017 = new NodeDLL_2511532017 (5);
    head_2017.next_2017.next_2017.prev_2017 = head_2017.next_2017;

    System.out.print("DLL Awal: ");
    printList_2017(head_2017);
    //tambah 1 di awal
    head_2017 = InsertBegin_2017 (head_2017, 1);
    System.out.print("Simpul 1 ditambah di awal : ");
    printList_2017 (head_2017);

    System.out.print(" simpul 6 ditambah di akhir: ");
    int data_2017 = 6;
    head_2017 = InsertEnd_2017 (head_2017, data_2017);
    printList_2017 (head_2017);

    System.out.print("tambah node 4 diposisi 4 : ");
    int data2_2017 = 4;
    int pos_2017 = 4;
    head_2017 = insertAtPosition_2017 (head_2017, pos_2017, data2_2017);
    printList_2017 (head_2017);

}
}

```

6. Method main, NodeDLL_2511532017 head_2017 = new NodeDLL_2511532017(2); inisialisasi, lalu dicetak menggunakan printList_2017 insertBegin insert diawal, insertEnd_2017 insert diakhir, insertAtPosition_2017 insert pada posisi tertentu dan semua dicetak menggunakan printList_2017

- HapusDLL_2511532017

1. kita akan membuat class baru untuk HapusDLL_2511532017

```

package pekan6_2511532017;

public class HapusDLL_2511532017 {
    public static NodeDLL_2511532017 delHead_2017 (NodeDLL_2511532017 head_2017) {
        if (head_2017 == null) {
            return null;
        }
        NodeDLL_2511532017 temp_2017 = head_2017;
        head_2017 = head_2017.next_2017;
        if(head_2017 != null) {
            head_2017.prev_2017 = null;
        }
        return head_2017;
    }
}

```

2. delHead, jika list kosong maka return null, temp_2017 = head_2017 untuk menyimpan ref head lama, head_2017 = head_2017.next_2017 untuk pindah ke node berikutnya, jika head tidak null maka putuskan koneksi prev ke node lama yang telah dihapus, return head_2017

```

public static NodeDLL_2511532017 delLast_2017 (NodeDLL_2511532017 head_2017) {
    if (head_2017 == null) {
        return null;
    }
    if (head_2017.next_2017 == null) {
        return null;
    }
    NodeDLL_2511532017 curr_2017 = head_2017;
    while (curr_2017.next_2017 != null) {
        curr_2017 = curr_2017.next_2017;
    }

    if (curr_2017.prev_2017 != null) {
        curr_2017.prev_2017.next_2017 = null;
    }
    return head_2017;
}

```

3. delLast_2017, jika list null maka return null, jika hanya ada satu node ,setelah dihapus list menjadi kosong maka return null, inisialisasi pointer baru untuk traversing, loop hingga menemukan node terakhir, putuskan koneksi node terakhir dan sekarang node kedua terkahir menjadi tail dan return head_2017

```

public static NodeDLL_2511532017 delPos_2017 (NodeDLL_2511532017 head_2017, int pos_2017) {
    if (head_2017 == null) {
        return head_2017;
    }

    NodeDLL_2511532017 curr_2017 = head_2017;
    for (int i = 1; curr_2017 != null && i < pos_2017; ++i) {
        curr_2017 = curr_2017.next_2017;
    }

    if (curr_2017 == null) {
        return head_2017;
    }
    if (curr_2017.prev_2017 != null) {
        curr_2017.prev_2017.next_2017 = curr_2017.next_2017;
    }
    if (curr_2017.next_2017 != null) {
        curr_2017.next_2017.prev_2017 = curr_2017.prev_2017;
    }
    if (head_2017 == curr_2017) {
        head_2017 = curr_2017.next_2017;
    }
    return head_2017;
}

```

4. delPos, jika list kosong return null, loop untuk menggeser pointer curr_2017 ke node pada posisi target, jika curr_2017= null, maka melebihi panjang list dan batalkan operasi, curr.prev.next = curr.next untuk hubungkan node sebelumnya ke node setelahnya, curr.next.prev = curr.prev hubungkan node setelahnya langsung ke node sebelumnya, jika node yang dihapus adalah head maka update head ke node berikutnya, return head_2017

```

public static void printList_2017 (NodeDLL_2511532017 head_2017) {
    NodeDLL_2511532017 curr_2017 = head_2017;
    while (curr_2017 != null) {
        System.out.print(curr_2017.data_2017+" ");
        curr_2017 = curr_2017.next_2017;
    }
    System.out.println();
}

```

5. printList_2017, mulai traversing dari node pertama, loop selama masih ada node yang valid, cetak data dengan spasi, pindah ke node berikutnya

```
public static void main (String [] args) {
    NodeDLL_2511532017 head_2017 = new NodeDLL_2511532017 (1);
    head_2017.next_2017 = new NodeDLL_2511532017 (2);
    head_2017.next_2017.prev_2017 = head_2017;
    head_2017.next_2017.next_2017 = new NodeDLL_2511532017 (3);
    head_2017.next_2017.next_2017.prev_2017 = head_2017.next_2017;
    head_2017.next_2017.next_2017.next_2017 = new NodeDLL_2511532017 (4);
    head_2017.next_2017.next_2017.next_2017.prev_2017 = head_2017.next_2017;
    head_2017.next_2017.next_2017.next_2017.next_2017 = new NodeDLL_2511532017 (5);
    head_2017.next_2017.next_2017.next_2017.next_2017.prev_2017 = head_2017.next_2017.next_2017;

    System.out.print("DLL awal: ");
    printList_2017(head_2017);

    System.out.print("setelah head dihapus: ");
    head_2017 = delHead_2017 (head_2017);
    printList_2017(head_2017);

    System.out.print("setelah node terakhir dihapus: ");
    head_2017 = delLast_2017 (head_2017);
    printList_2017(head_2017);

    System.out.print("menghapus node ke 2: ");
    head_2017 = delPos_2017 (head_2017, 2);
    printList_2017 (head_2017);
}
```

6. method main , inialisasi dan cetak dengan printList_2017, delHead_2017 untuk menghapus head, delLast_2017 untuk menghapus node terakhir, delPos_2017 untuk menghapus node pada posisi tertentu lalu cetak semua dengan printList_2017

- PenelusuranDLL_2511532017

1. Buat class dengan nama PenelusuranDLL_2511532017

```
package pekan6_2511532017;

public class PenelusuranDLL_2511532017 {

    static void forwardTraversal_2017 (NodeDLL_2511532017 head_2017) {
        NodeDLL_2511532017 curr_2017 = head_2017;
        while (curr_2017 != null) {
            System.out.print(curr_2017.data_2017 + " <--> ");
            curr_2017 = curr_2017.next_2017;
        }

        System.out.println();
    }
}
```

2. forwardTraversal_2017, curr_2017 = head_2017 untuk inialisasi pointer bantu mulai dari node pertama, loop selama masih ada node yang valid, lalu cetak data dengan tanda <-->, curr_2017 = curr_2017.next_2017 geser pointer ke node berikutnya

```
static void backwardTraversal_2017 (NodeDLL_2511532017 tail_2017) {
    NodeDLL_2511532017 curr_2017 = tail_2017;
    while (curr_2017 != null) {
        System.out.print(curr_2017.data_2017 + " <--> ");
        curr_2017 = curr_2017.prev_2017;
    }
    System.out.println();
}
```

- backwardTraversal_2017, mulai traversing dari node terakhir, loop selama masih node masih valid , curr_2017 = curr_2017.prev_2017 geser pointer mundur menggunakan prev

```
public static void main (String [] args) {
    NodeDLL_2511532017 head_2017 = new NodeDLL_2511532017 (1);
    NodeDLL_2511532017 second_2017 = new NodeDLL_2511532017 (2);
    NodeDLL_2511532017 third_2017 = new NodeDLL_2511532017 (3);

    head_2017.next_2017 = second_2017;
    second_2017.prev_2017 = head_2017;
    second_2017.next_2017 = third_2017;
    third_2017.prev_2017 = second_2017;

    System.out.println("penelusuran maju: ");
    forwardTraversal_2017(head_2017);

    System.out.println("penelusuran mundur: ");
    backwardTraversal_2017 (third_2017);
}
}
```

- method main, NodeDLL_2511532017 head_2017 = new NodeDLL_2511532017(1); membaut node baru, head_2017.next_2017 = second untuk mengarahkan next node satu ke node dua, second_2017.prev_2017 = head_2017 arahkan prev node dua ke node satu, lalu print forwardTraversal_2017 dan backwardTraversal

Latihan

- Lagu_2511532017
 - Kita akan membuat class baru untuk Lagu_2511532017

```
package pekan6_2511532017;

import pekan5_2511532017.Pasien_2511532017;

public class Lagu_2511532017 {
    private String judul_2017;
    private String penyanyi_2017;
    private Lagu_2511532017 next_2017;
    private Lagu_2511532017 prev_2017;
}
```

- Deklarasi class dan atribut, private String judul_2017 untuk menyimpan judul, penyanyi_2017 untuk menyimoan nama penyanyi, next_2017 untuk pointer selanjutnya dan prev_2017 untuk ke node sebelumnya

```
//constructor
public Lagu_2511532017 (String judul_2017, String penyanyi_2017) {
    this.judul_2017 = judul_2017;
    this.penyanyi_2017 = penyanyi_2017;

}
```

3. Constructor, method khusus yang dipanggil saat membuat objek Lagu_2511532017 baru, menginisialisasi judul dan penyanyi

```
//setter getter
public String getJudul_2017() {return judul_2017;}
public void setJudul_2017(String judul_2017) {this.judul_2017= judul_2017;}

public String getPenyanyi_2017 () { return penyanyi_2017;}
public void setPenyanyi_2017 (String penyanyi_2017) {this.penyanyi_2017 = penyanyi_2017;}

public Lagu_2511532017 getNext_2017 () {return next_2017;}
public void setNext_2017 (Lagu_2511532017 next_2017) {this.next_2017 = next_2017;}

public Lagu_2511532017 getPrev_2017 () {return prev_2017;}
public void setPrev_2017 (Lagu_2511532017 prev_2017) {this.prev_2017 = prev_2017;}
```

4. Getter untuk mengambil nilai atribut private diluar kelas dan setter untuk mengubah nilai atribut private di luar kelas

```
public static Lagu_2511532017 findTail_2017(Lagu_2511532017 head_2017) {
    if (head_2017 == null) return null;
    Lagu_2511532017 curr_2017 = head_2017;
    while (curr_2017.getNext_2017() != null) {
        curr_2017 = curr_2017.getNext_2017();
    }
    return curr_2017;
}
```

5. findTail_2017 digunakan untuk parameter tail agar pencarian tail dinamis

```
public static Lagu_2511532017 tambahLaguAkhir_2017 (Lagu_2511532017 head_2017, String lagu_2017, String penyanyi_2017) {
    //buat node baru
    Lagu_2511532017 newNode_2017 = new Lagu_2511532017 (lagu_2017, penyanyi_2017);
    //jika dll null jadikan head
    if (head_2017 == null) {
        head_2017 = newNode_2017;
    }
    else {
        Lagu_2511532017 curr_2017 = head_2017;
        while (curr_2017.next_2017 != null) {
            curr_2017 = curr_2017.next_2017;
        }
        curr_2017.next_2017 = newNode_2017;
        newNode_2017.prev_2017 = curr_2017;
    }
    return head_2017;
}
```

6. tambahLaguAkhir_2017, newNode_2017 berisi dari data lagu baru, jika head kosong maka node menjadi head, jika tidak kosong maka traversal sampai ke node terakhir curr_2017.next_2017 == null). Lalu hubungkan dengan curr_2017.next_2017 = newNode_2017 dan hubungkan balik dengan newNode_2017.prev_2017 = curr_2017 dan return head_2017

```

public static Lagu_2511532017 delHead_2017 (Lagu_2511532017 head_2017) {
    if (head_2017 == null) {
        System.out.println("playlist masih kosong");
        return null;
    }
    Lagu_2511532017 temp_2017 = head_2017;
    head_2017 = head_2017.next_2017;
    if (head_2017 != null) {
        head_2017.prev_2017 = null;
        System.out.println("Lagu berhasil dihapus");
    }
    return temp_2017;
}

```

7. delHead_2017, cek jika list kosong cetak pesan kosong dan return null, simpan head sementara di temp_2017, pindahkan head ke node berikutnya, jika node baru tidak null putuskan link prev nya, return temp_2017

```

static void forwardTraversal_2017 (Lagu_2511532017 head_2017) {
    Lagu_2511532017 curr_2017 = head_2017;
    while (curr_2017 != null) {
        System.out.print(curr_2017.toString());
        System.out.println();
        curr_2017 = curr_2017.next_2017;
    }
    System.out.println();
}

```

8. forwardTraversal_2017, mulai dari depan head_2017, loop selama curr_2017 != null, cetak node menggunakan toString(), pindahkan ke node berikutnya, curr_2017 = curr_2017.next_2017

```

static void backwardTraversal_2017 (Lagu_2511532017 tail_2017) {
    Lagu_2511532017 curr_2017 = tail_2017;
    while (curr_2017 != null) {
        System.out.print(curr_2017.toString());
        System.out.println();
        curr_2017 = curr_2017.prev_2017;
    }
    System.out.println();
}

```

9. backwardTraversal_2017, mulai dari tail_2017, loop selama curr_2017 != null, cetak data node dan pindah ke node sebelumnya dengan curr_2017 = curr_2017.prev_2017

```

static boolean cariJudul_2511532017 (Lagu_2511532017 head_2017, String key_2017) {
    Lagu_2511532017 curr_2017 = head_2017;
    while (curr_2017 != null) {
        if (curr_2017.judul_2017.equalsIgnoreCase(key_2017)) {
            System.out.println();
            System.out.println("judul lagu ditemukan");
            System.out.println(curr_2017.toString());
            return true;
        }
        curr_2017 = curr_2017.next_2017;
    }
    System.out.println();
    System.out.println("Judul Lagu " + key_2017 + " tidak ditemukan");
    return false;
}

```

10. cariJudul_2017, traversal dari head ke akhir, bandingkan judul_2017 dengan key_2017 menggunakan equalsIgnoreCase(), jika ditemukan cetak pesan ditemukan dan judul lagu return true, jika tidak ditemukan cetak pesan tidak ditemukan

```
@Override
public String toString() {
    return "\nJudul Lagu: " + judul_2017 +
        "\nPenyanyi: " + penyanyi_2017;
}
```

11. toString, method khusus yang otomatis dipanggil saat objek dicetak

- Musik_2511532017

1. Kita buat kelas untuk Musik_2511532017

```
package pekanb_2511532017;
import java.util.Scanner;
public class Musik_2511532017 {
    // Method untuk menampilkan menu
    public static void tampilkanMenu_2511532017() {
        System.out.println("\n=== Playlist Musik NIM: 2511532017 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
    }
}
```

2. Method tampilkan menu memberikan 6 pilihan menu

```
do {
    tampilkanMenu_2511532017();
    System.out.print("Pilih menu (1-6): ");

    try {
        pilihan = Integer.parseInt(sc.nextLine().trim());
    } catch (NumberFormatException e) {
        pilihan = -1;
        System.out.println("Input harus angka 1-6!");
        continue;
    }
}
```

3. Inisialisai variabel, dan struktur loop do while yang menjamin menu akan tampil minimal satu kali sebelum pengecekan kondisi

```
do {
    tampilkanMenu_2511532017();
    System.out.print("Pilih menu (1-6): ");

    try {
        pilihan = Integer.parseInt(sc.nextLine().trim());
    } catch (NumberFormatException e) {
        pilihan = -1;
        System.out.println("Input harus angka 1-6!");
        continue;
    }
}
```

4. Method menu handling `sc.nextLine()` membaca seluruh baris input sebagai string, `trim()` menghapus spasi dan `Integer.parseInt()` mengkonversi String menjadi Integer. Jika input bukan angka valid maka `NumberFormatException`
5. Case 1 input judul, cek apakah judul kosong, jika valid input penyanyi, panggil `tambahLaguAkhir_2017()`. Update `head_2017`
6. Case 2 hapus lagu pertama dengan memanggil method `delHead_2017` dan update `head_2017`
7. Case 3 lihat playlist maju, cek kosong atau tidak dan panggil `forwardTraversal_2017()`
8. Case 4 lihat apakah kosong atau tidak, cari tail menggunakan helper method `findTail_2017()` dan panggil method `backwardTraversal_2017`
9. Case 5 cari lagu, input judul, panggil method `cariJudul_2017`, traversal sampai ketemu
10. Case 6, keluar dari menu dan menampilkan pesan penutup
11. Output

Input judul

```
=== Playlist Musik NIM: 2511532017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 1
Masukkan judul lagu: the Middle
Masukkan nama penyanyi: Zedd
Lagu berhasil ditambahkan!
```

Lihat playlist maju

```

=== Playlist Musik NIM: 2511532017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 3
|
Judul Lagu: the Middle
Penyanyi: Zedd

Judul Lagu: die very rough
Penyanyi: mario judah

Judul Lagu: outside
Penyanyi: calvin harris

```

Lihat playlist mundur

```

=== Playlist Musik NIM: 2511532017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 4

Judul Lagu: outside
Penyanyi: calvin harris

Judul Lagu: die very rough
Penyanyi: mario judah

Judul Lagu: the Middle
Penyanyi: Zedd

```

Cari lagu

```

=== Playlist Musik NIM: 2511532017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilih menu (1-6): 5
Masukkan judul lagu yang dicari: die very rough

judul lagu ditemukan

Judul Lagu: die very rough
Penyanyi: mario judah

```

Hapus lagu pertama

```
=== Playlist Musik NIM: 2511532017 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilih menu (1-6): 2  
Lagu berhasil dihapus
```

Setelah dihapus

```
=== Playlist Musik NIM: 2511532017 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilih menu (1-6): 3  
  
Judul Lagu: die very rough  
Penyanyi: mario judah  
  
Judul Lagu: outside  
Penyanyi: calvin harris
```

BAB III

KESIMPULAN

Struktur data double LinkedList berhasil diimplementasikan dengan baik. Melalui praktikum ini, pemahaman mengenai konsep Doubly Linked List, manajemen pointer next dan prev. Playlist musik sederhana berhasil dibuat dengan menggunakan pointer next dan prev. Program ini mengimplementasikan operasi fundamental seperti penambahan lagu di akhir list, penghapusan lagu dari posisi pertama, traversing dua arah (maju dan mundur), serta pencarian lagu berdasarkan judul.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Linked list data structure," GeeksforGeeks.
<https://www.geeksforgeeks.org/linked-list-data-structure/>
(accessed May 13, 2026).